

Freestanding Library: Partial Classes

Document number: P2407R5

Date: 2023-07-26

Reply-to:

Ben Craig <ben dot craig at gmail dot com>

Emil Meissner <e dot meissner at seznam dot cz>

Audience: Library Working Group

Changes from previous revisions

Changes from R4

- Clarified "a member of a freestanding item" to non-namespace members
- Explicit renumbering in `freestanding.item` to make conversation possible
- Tweaked requirements on freestanding items to not require all members, with later rules putting most of those requirements back
- Discussing entities rather than function definitions
- Simplified freestanding comment grammar. Now the only normative words are `freestanding` and `freestanding-deleted`
- Tweaked notes on freestanding to hosted migration
- Added some drafting and LWG notes explaining wording rationale
- Using exposition-only `GET` function in `variant` rather than `respecify` in terms of `get_if`
- Using public interface of `string_view` rather than exposition interface when accessing non-this objects

Changes from R3

- Added `fill_n` and `swap_ranges` from `<algorithm>`
- `optional` now specified in terms of `**this` instead of `.value()`
- `variant` now specified in terms of `get_if` instead of `get`
- `string_view` `find` / `search` operations now specified in terms of `data_` instead of `at()`
- Added missing parenthesis on `size()` call in `string_view::ends_with`
- Made feature test macros `// freestanding`
- More deliberate with use of "synopsis" vs. "header synopsis"
- `freestanding-deleted` and `freestanding-partial` are now freestanding items with altered requirements
- Added notes stating overload resolution consequences of `// hosted` vs. `// freestanding-deleted`
- More explicitly making the members of freestanding items also be freestanding items
- Deleted note discussing implication of having the same requirements, as those are covered more explicitly now

Changes from R2

- `// freestanding-delete` to `// freestanding-deleted`
- Adding `bad_optional_access` per LWG's request for consistency
- Rewording `string_view::starts_with` and `string_view::ends_with` in terms of freestanding functions
- Avoiding word "item" in instructions to the editor
- Made example in wording use `// freestanding-deleted`
- Added `// mostly freestanding` header marker

Changes from R1

- Ported paper to HTML
- Only annotating the deleted parts of partial classes, and not the included parts
- Mentioning `forward_like` in relation to variant's value categories
- New prose for `[freestanding.item]`

Changes from R0

- Add wording for feature test macros
- Mention monadic optional and `string_view::contains`

Introduction

This proposal is part of a group of papers aimed at improving the state of freestanding. It marks (parts of) `std::array`, `std::string_view`, `std::variant`, and `std::optional` as such. A future paper might add `std::bitset` (as was the original goal in `[P2268R0]`)

Motivation and Scope

All of the added classes are fundamentally compatible with freestanding, except for a few methods that throw (e.g. `array::at`). We explicitly `=delete` these undesirable methods.

The main driving factor for these additions is the immense usefulness of these types in practice.

Scope

We refine `[freestanding.item]` by specifying the notion of partial classes, and accordingly specify the newly (partially) freestanding classes as such.

About `<bitset>`

As mentioned in the introduction, this paper does not deal with `bitset`. `Bitset` is unique in that a relatively big part of its interface depends on `std::basic_string`. We do not currently have a sound plan to make `bitset` work as nicely as we'd like to. This situation is made worse by a significant amount of `bitset`'s member functions that throw.

Implementation experience

The Existing Standard Library

We've forked `libc++`, and `=deleted` all not freestanding methods. Except for some methods on `string_view` (which are implemented in terms of the deleted `string_view::substring`), this did not require any changes in the implementation. All test cases (except for the deleted methods) passed after some rather minor adjustments (e.g. replacing `get<0>(v)` with `*get_if<0>(&v)`), confirming that all these types are usable without the deleted methods.

In Practice

Since we aren't changing the semantics of any of the classes (except deleted non-critical methods), it is fair to say that all of the (implementer and user) experience gathered as part of hosted applies the same to freestanding.

The only question is, whether these classes are compatible with freestanding. To which the answer is yes! For example, the [Embedded Template Library] offers direct mappings of the std types. Even in kernel-level libraries, like Serenity's [AK] use a form of these utilities.

Design decisions

Deleting behavior

Our decision to delete methods we can't mark as freestanding was made to keep overload resolution the same on freestanding as hosted.

An additional benefit here is, that users of these classes, who might expect to use a throwing method, which was not provided by the implementation, will get a more meaningful error than the method simply missing. This also means we can keep options open for reintroducing the deleted functions into freestanding. (e.g. `operator<<(ostream, string_view)`, should `<ostream>` be added).

[conventions] changes

The predecessor to this paper used `//freestanding, partial` to mean a class (template) is only required to be partially implemented, in conjunction with `//freestanding, omit` meaning a declaration is not in freestanding.

In this paper, we mark not fully freestanding classes templates as `// partially freestanding`, and use P2338's `// freestanding-deleted` to mark which pieces of the class should be omitted. We no longer annotate all the class members, favoring terseness over explicitness.

On `std::visit`

In this paper, we mark `std::visit` as freestanding, even though it is theoretically throwing. However, the conditions for `std::visit` to throw are as follows:

It is possible for a variant to hold no value if an exception is thrown during a type-changing assignment or emplacement.

This means a variant will only throw on visit if a user type throws (library types don't throw on freestanding). In this case, `std::visit` throwing isn't a problem, since the user's code is already using, and (hopefully) handling exceptions.

This however has the unfortunate side-effect that we need to keep `bad_variant_access` freestanding.

Notes on variant and value categories

By getting rid of `std::get`, we force users to use `std::get_if`. Since `std::get_if` returns a pointer, one can only access the value of a variant by dereferencing said pointer, obtaining an lvalue, discarding the value category of the held object. This is unlikely to have an impact on application code, but might impact highly generic library code.

`std::forward_like` can help in these cases. The value category of the variant can be transferred to the dereferenced pointer returned from `std::get_if`.

Justification for deletions

Every deleted method is throwing. We omit `string_view`'s associated operator<< since we don't add `basic_ostream`.

Monadic optional and `string_view::contains`

Since this paper was first published, `std::string_view` got a new `contains` member function, and `std::optional` got `transform`, `and_then`, and `or_else`. All these functions are not throwing, and there are no other problems regarding freestanding. We therefore opt for them being marked as freestanding.

Minimal `<algorithm>` inclusions

`std::array::fill` is specified in terms of `std::fill_n`. `std::array::swap` is specified in terms of `std::swap_ranges`. Both `fill_n` and `swap_ranges` are reasonable inclusions in freestanding. Rather than respecify `array` in terms of other freestanding facilities, we have chosen to pull in the functions from `<algorithm>` that we need. A later paper will likely add many more facilities from `<algorithm>`. An email on the LEWG reflector in June 2023 asked to add `std::fill_n` and `std::swap_ranges`. The request received eleven +1's, and no opposition to this change. The authors did not vote on the reflector poll.

Wording

This paper's wording is based on the working draft, [N4950], plus the anticipated inclusion of merging [P2198R6] (Freestanding Feature-Test Macros and Implementation-Defined Extensions) and [P2338R4] (Freestanding Library: Character primitives and the C library).

Change in [freestanding.item]

Modify [freestanding.item]. This includes a reordering of what was paragraph 6.

- 2 Unless otherwise specified, the requirements on freestanding items for a freestanding implementation are the same as the corresponding requirements for a hosted implementation, except that not all of the members of ~~the namespaces~~ **those items** are required to be present.

~~[Note 1: This implies that freestanding item enumerations have the same enumerators on freestanding implementations and hosted implementations. Furthermore, class types have the same members and class templates have the same deduction guides on freestanding implementations and hosted implementations. —end note]~~

- 63 Function declarations and function template declarations followed by a comment that include *freestanding-deleted* are *freestanding deleted functions*. On freestanding implementations, it is implementation defined whether each ~~function definition~~ **entity** introduced by a freestanding deleted function is ~~a freestanding item or~~ a deleted function ([dcl.fct.def.delete]) **or whether the requirements are the same as the corresponding requirements for a hosted implementation.**

[Note: Deleted definitions reduce the chance of overload resolution silently changing when migrating from a freestanding implementation to a hosted implementation. -end note]

[Example:

double abs(double j); //freestanding-deleted

-end example]

- 34 A declaration in a ~~header~~ synopsis is a freestanding item if

(3.1)(4.1) • it is followed by a comment that includes *freestanding*, ~~or~~

- (4.2) • it is followed by a comment that includes *freestanding-deleted*, or
- (3.2)(4.3) • the header synopsis begins with a comment that includes ~~all~~ *freestanding* and the declaration is not followed by a comment that includes *hosted*.
[*Note: Declarations followed by *hosted* in *freestanding* headers are not freestanding items. As a result, looking up the name of such functions can vary between hosted and freestanding implementations. -end note*]

45 An entity, **deduction guide**, or *typedef-name* is a freestanding item if it is:

- (4.1)(5.1) • introduced by a declaration that is a freestanding item,
- (5.2) • a member of a freestanding item other than a namespace,
- (5.3) • an enumerator of a freestanding item,
- (5.4) • a deduction guide of a freestanding item,
- (4.2)(5.5) • an enclosing namespace of a freestanding item,
- (4.3)(5.6) • a friend of a freestanding item,
- (4.4)(5.7) • denoted by a *typedef-name* that is a freestanding item, or
- (4.5)(5.8) • denoted by an alias template that is a freestanding item.

56 A macro is a freestanding item if it is defined in a header synopsis and

- (5.1)(6.1) • the definition is followed by a comment that includes *freestanding*, or
- (5.2)(6.2) • the header synopsis begins with a comment that includes ~~all~~ *freestanding* and the definition is not followed by a comment that includes *hosted*.

7 [*Note: Freestanding annotations follow some additional exposition conventions that do not impose any additional normative requirements. Header synopses that begin with a comment containing "all freestanding" contain no hosted items and no freestanding deleted functions. Header synopses that begin with a comment containing "mostly freestanding" contain at least one hosted item or freestanding deleted function. Classes and class templates followed by a comment containing "partially freestanding" contain at least one hosted item or freestanding deleted function. -end note*]

[*Example:*

```
template <class T, size_t N> struct array; //partially freestanding

template<class T, size_t N>
struct array {
    constexpr reference      operator[](size_type n);
    constexpr const_reference operator[](size_type n) const;
    constexpr reference      at(size_type n); //freestanding-deleted
    constexpr const_reference at(size_type n) const; //freestanding-deleted
};
```

-end example]

Changes in [compliance]

Add new rows to the "C++ headers for freestanding implementations" table:

Subclause		Header(s)
[...]	[...]	[...]
?? [optional]	Optional objects	<optional>
?? [variant]	Variants	<variant>
?? [string.view]	String view classes	<string_view>

Subclause	Header(s)
?? [array]	Class template array <array>
?? [algorithms]	Algorithms library <algorithm>
[...]	[...]

Changes in [\[version.syn\]](#)

This part of the paper follows the guide lines as specified in [P2198R6]. Instructions to the editor:
Add the following macros to [\[version.syn\]](#):

```
#define cpp_lib_freestanding_algorithm 20XXXXL // freestanding, also in <algorithm>
#define cpp_lib_freestanding_array 20XXXXL // freestanding, also in <array>
#define cpp_lib_freestanding_optional 20XXXXL // freestanding, also in <optional>
#define cpp_lib_freestanding_string_view 20XXXXL // freestanding, also in <string_view>
#define cpp_lib_freestanding_variant 20XXXXL // freestanding, also in <variant>
```

Changes in [\[optional.syn\]](#)

Instructions to the editor:

Please insert a *// mostly freestanding* comment at the beginning of the [\[optional.syn\]](#) synopsis.

Please append a *// partially freestanding* comment to the following declaration:

- optional

Changes in [\[optional.optional.general\]](#)

Instructions to the editor:

Please append a *// freestanding-deleted* comment to every overload of value.

Changes in [\[optional.monadic\]](#)

Please modify the functions in [\[optional.monadic\]](#) so that they don't reference the *freestanding-deleted* method value().

```
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) const &;
1      Let U be invoke_result_t<F, decltype (value()**this)>.
2      Mandates: remove_cvref_t<U> is a specialization of optional.
3      Effects: Equivalent to:
      if (*this) {
          return invoke(std::forward<F>(f), value()**this);
      } else {
          return remove_cvref_t<U>();
      }
```

```
template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &&;
```

Let U be `invoke_result_t<F, decltype(std::move(value()**this))>`.

Mandates: `remove_cvref_t<U>` is a specialization of `optional`.

Effects: Equivalent to:

```
if (*this) {
    return invoke(std::forward<F>(f), std::move(value()**this));
} else {
    return remove_cvref_t<U>();
}
```

```
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &&;
```

Let U be `remove_cv_t<invoke_result_t<F, decltype(value()**this)>>`.

Mandates: U is a non-array object type other than `in_place_t` or `nullopt_t`. The declaration

```
U u(invoke(std::forward<F>(f), value()**this));
```

is well-formed for some invented variable u.

[Note 1: There is no requirement that U is movable ([[dcl.init.general](#)]). — end note]

Returns: If `*this` contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), value()**this)`; otherwise, `optional<U>()`.

```
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &&;
```

Let U be `remove_cv_t<invoke_result_t<F, decltype(std::move(value()**this))>>`.

Mandates: U is a non-array object type other than `in_place_t` or `nullopt_t`. The declaration

```
U u(invoke(std::forward<F>(f), std::move(value()**this)));
```

is well-formed for some invented variable u.

[Note 2: There is no requirement that U is movable ([[dcl.init.general](#)]). — end note]

Returns: If `*this` contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(value()**this))`; otherwise, `optional<U>()`.

Changes in [[variant.general](#)]

LWG note: four *GET* overloads are added, because we can't use `get` in the specification anymore since it could be `=deleted`, and we still need the right return types, value categories, and mandates of *GET* to deal with `visit` and the `relops`.

Add a new paragraph to [[variant.general](#)].

In subclause [variant], *GET* denotes a set of exposition only function templates ([variant.get]).

Changes in [variant.syn]

```
// mostly freestanding
#include <compare> // see [compare.syn]

namespace std {
```

...

```
// [variant.get], value access
template<class T, class... Types>
    constexpr bool holds_alternative(const variant<Types...>&) noexcept;

template<size_t I, class... Types>
    constexpr variant_alternative_t<I, variant<Types...>>& get(variant<Types...>&);
template<size_t I, class... Types>
    constexpr variant_alternative_t<I, variant<Types...>>&& get(variant<Types...>&&);
template<size_t I, class... Types>
    constexpr const variant_alternative_t<I, variant<Types...>>& get(const variant<Types...>&);
template<size_t I, class... Types>
    constexpr const variant_alternative_t<I, variant<Types...>>&& get(const variant<Types...>&&);

template<class T, class... Types>
    constexpr T& get(variant<Types...>&); // freestanding-deleted
template<class T, class... Types>
    constexpr T&& get(variant<Types...>&&); // freestanding-deleted
template<class T, class... Types>
    constexpr const T& get(const variant<Types...>&); // freestanding-deleted
template<class T, class... Types>
    constexpr const T&& get(const variant<Types...>&&); // freestanding-deleted
```

Changes in [variant.ctor]

```
constexpr variant(const variant& w);
```

7 *Effects:* If *w* holds a value, initializes the variant to hold the same alternative as *w* and direct-initializes the contained value with `getGET<j>(w)`, where *j* is *w.index()*. Otherwise, initializes the variant to not hold a value.

8 *Throws:* Any exception thrown by direct-initializing any *T_i* for all *i*.

9 *Remarks:* This constructor is defined as deleted unless `is_copy_constructible_v<Ti>` is true for all *i*. If `is_trivially_copy_constructible_v<Ti>` is true for all *i*, this constructor is trivial.

```
constexpr variant(variant&& w) noexcept(see below);
```

10 *Constraints:* `is_move_constructible_v<Ti>` is true for all *i*.

11 *Effects:* If *w* holds a value, initializes the variant to hold the same alternative as *w* and direct-initializes the contained value with `get<GET><j>(std::move(w))`, where *j* is `w.index()`. Otherwise, initializes the variant to not hold a value.

Changes in `[variant.assign]`

```
constexpr variant& operator=(const variant& rhs);
```

1 Let *j* be `rhs.index()`.

2 *Effects:*

- (2.1) • If neither `*this` nor *rhs* holds a value, there is no effect.
- (2.2) • Otherwise, if `*this` holds a value but *rhs* does not, destroys the value contained in `*this` and sets `*this` to not hold a value.
- (2.3) • Otherwise, if `index() == j`, assigns the value contained in *rhs* to the value contained in `*this`.
- (2.4) • Otherwise, if either `is_nothrow_copy_constructible_v<Tj>` is true or `is_nothrow_move_constructible_v<Tj>` is false, equivalent to `emplace<j>(get<GET><j>(rhs))`.
- (2.5) • Otherwise, equivalent to `operator=(variant(rhs))`.

3 *Postconditions:* `index() == rhs.index()`.

4 *Returns:* `*this`.

5 *Remarks:* This operator is defined as deleted unless `is_copy_constructible_v<Ti> && is_copy_assignable_v<Ti>` is true for all *i*. If `is_trivially_copy_constructible_v<Ti> && is_trivially_copy_assignable_v<Ti> && is_trivially_destructible_v<Ti>` is true for all *i*, this assignment operator is trivial.

```
constexpr variant& operator=(variant&& rhs) noexcept(see below);
```

6 Let *j* be `rhs.index()`.

7 *Constraints:* `is_move_constructible_v<Ti> && is_move_assignable_v<Ti>` is true for all *i*.

8 *Effects:*

- (8.1) • If neither `*this` nor *rhs* holds a value, there is no effect.
- (8.2) • Otherwise, if `*this` holds a value but *rhs* does not, destroys the value contained in `*this` and sets `*this` to not hold a value.
- (8.3) • Otherwise, if `index() == j`, assigns `get<GET><j>(std::move(rhs))` to the value contained in `*this`.
- (8.4) • Otherwise, equivalent to `emplace<j>(get<GET><j>(std::move(rhs)))`.

Changes in `[variant.swap]`

```
constexpr void swap(variant& rhs) noexcept(see below);
```

Mandates: `is_move_constructible_v<Ti>` is true for all *i*.

Preconditions: Each `Ti` meets the `Cpp17Swappable` requirements ([\[swappable.requirements\]](#)).

Effects:

- (3.1) • If `valueless_by_exception()` && `rhs.valueless_by_exception()` no effect.
- (3.2) • Otherwise, if `index() == rhs.index()`, calls `swap(get<GET><i>(*this), get<GET><i>(rhs))` where *i* is `index()`.
- (3.3) • Otherwise, exchanges values of `rhs` and `*this`.

Throws: If `index() == rhs.index()`, any exception thrown by `swap(get<GET><i>(*this), get<GET><i>(rhs))` with *i* being `index()`. Otherwise, any exception thrown by the move constructor of `Ti` or `Tj` with *i* being `index()` and *j* being `rhs.index()`.

Remarks: If an exception is thrown during the call to function `swap(get<GET><i>(*this), get<GET><i>(rhs))`, the states of the contained values of `*this` and of `rhs` are determined by the exception safety guarantee of `swap` for lvalues of `Ti` with *i* being `index()`. If an exception is thrown during the exchange of the values of `*this` and `rhs`, the states of the values of `*this` and of `rhs` are determined by the exception safety guarantee of `variant`'s move constructor. The exception specification is equivalent to the logical AND of `is_nothrow_move_constructible_v<Ti>` && `is_nothrow_swappable_v<Ti>` for all *i*.

Changes in [\[variant.relops\]](#)

Instructions to the editor:

Replace every instance of `get<i>` in [\[variant.relops\]](#) with `GET<i>`.

Changes in [\[variant.get\]](#)

```
template<size_t I, class... Types>
constexpr variant_alternative_t<I, variant<Types...>& GET(variant<Types...>& v); //
template<size_t I, class... Types>
constexpr variant_alternative_t<I, variant<Types...>&& GET(variant<Types...>&& v);
template<size_t I, class... Types>
constexpr const variant_alternative_t<I, variant<Types...>& GET(const variant<Types...>& v);
template<size_t I, class... Types>
constexpr const variant_alternative_t<I, variant<Types...>&& GET(const variant<Types...>&& v);
```

Mandates: `I < sizeof...(Types)`.

Preconditions: `v.index()` is *I*.

Returns: A reference to the object stored in the variant.

```
template<size_t I, class... Types>
constexpr variant_alternative_t<I, variant<Types...>& get(variant<Types...>& v);
template<size_t I, class... Types>
constexpr variant_alternative_t<I, variant<Types...>&& get(variant<Types...>&& v);
template<size_t I, class... Types>
```

```
constexpr const variant_alternative_t<I, variant<Types...>>& get(const variant<Types...>& v,
template<size_t I, class... Types>
constexpr const variant_alternative_t<I, variant<Types...>>&& get(const variant<Types...>& v,
```

36 Mandates: $I < \text{sizeof} \dots (\text{Types})$.

47 Effects: If `v.index()` is `I`, returns a reference to the object stored in the variant. Otherwise, throws an exception of type `bad_variant_access`.

Changes in [\[variant.visit\]](#)

4 Let m be a pack of n values of type `size_t`. Such a pack is valid if $0 \leq m_i < \text{variant_size_v} \langle \text{remove_reference_t} \langle V_i \rangle \rangle$ for all $0 \leq i < n$. For each valid pack m , let $e(m)$ denote the expression:

```
INVOKE(std::forward<Visitor>(vis), get<GET><m>(std::forward<V>(vars))...) // see [
```

for the first form and

```
INVOKE<R>(std::forward<Visitor>(vis), get<GET><m>(std::forward<V>(vars))...) // s
```

for the second form.

Changes in [\[string.view.synop\]](#)

Instructions to the editor:

Please insert a `// mostly freestanding` comment at the beginning of the [\[string.view.synop\]](#) synopsis.

Please append a `// hosted` comment to the following declaration:

- `operator<<`

Please append a `// partially freestanding` comment to the following declaration:

- `basic_string_view`

Changes in [\[string.view.template.general\]](#)

Instructions to the editor:

Please append a `// freestanding-deleted` to the following functions:

- `at`
- `copy`
- `substr`
- The following overloads of `compare`:
 - `compare(size_type pos1, size_type n1, basic_string_view s)`
 - `compare(size_type pos1, size_type n1, basic_string_view s, size_type pos2, size_type n2)`
 - `compare(size_type pos1, size_type n1, const charT* s)`
 - `compare(size_type pos1, size_type n1, const charT* s, size_type n2)`

Note that the `compare(basic_string_view str) const` and `compare(const charT* s) const` overloads are intentionally not freestanding-deleted.

Changes in `[string.view.ops]`

Please modify the `basic_string_view` overload of `starts_with` so that it doesn't reference *freestanding-deleted* methods.

```
constexpr bool starts_with(basic_string_view x) const noexcept;
```

? **Let `rlen` be the smaller of `size()` and `x.size()`.**
? **Effects:** Equivalent to: ~~`return substr(0, x.size()) == x;`~~ **`return`**
`basic_string_view(data(), rlen) == x;`

Please modify the `basic_string_view` overload of `ends_with` so that it doesn't reference *freestanding-deleted* methods.

```
constexpr bool ends_with(basic_string_view x) const noexcept;
```

? **Let `rlen` be the smaller of `size()` and `x.size()`.**
? **Effects:** Equivalent to: ~~`return size() >= x.size() && compare(size() - x.size(), npos,`~~
~~`x) == 0;`~~ **`return basic_string_view(data() + (size() - rlen), rlen) == x;`**

Changes in `[string.view.find]`

Please avoid using the *freestanding-deleted* method `at()`.

Instructions to the editor:

- Replace every instance of `at(xpos + I)` in `[string.view.find]` with `data_[xpos + I]`.
- Replace every instance of `at(xpos)` in `[string.view.find]` with `data_[xpos]`.
- Replace every instance of `str.at(I)` in `[string.view.find]` with `str[I]`.

Changes in `[array.syn]`

Instructions to the editor:

Please insert a *// mostly freestanding* comment at the beginning of the `[array.syn]` synopsis.

Please append a *// partially freestanding* comment to `array`

Changes in `[array.overview]`

Instructions to the editor:

Please append a *// freestanding-deleted* comment to every overload of `at`.

Changes in `[algorithm.syn]`

Instructions to the editor:

Please append a `// freestanding` comment to the following functions, so that `std::array` can be specified in terms of freestanding functions:

- `fill_n(OutputIterator first, Size n, const T& value)`
- `swap_ranges(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2)`

References

[AK] Andreas Kling. Serenity OS AK Library.

<https://github.com/SerenityOS/serenity/tree/master/AK>

[Embedded Template Library] John Wellbelove. Embedded Template Library.

<https://www.etlcpp.com/>

[N4928] Thomas Köppe. 2022-12-18. Working Draft, Standard for Programming Language C++.

<https://wg21.link/n4928>

[P2198R6] Ben Craig. 2022-12-06. Freestanding Feature-Test Macros and Implementation-Defined Extensions.

<https://wg21.link/P2198R6>

[P2268R0] Ben Craig. 2020-12-10. Freestanding Roadmap.

<https://wg21.link/p2268r0>

[P2338R4] Ben Craig. 2023-02-09. Freestanding Library: Character primitives and the C library.

<https://wg21.link/P2338R4>